[디버깅] 메모리 누수를 점검하는 방법 - 성능 프로파일러

2020.04.17 21:38 Programming/Debug...

메모리 누수와 관련된 포스팅은 아래 링크를 참고해주세요.

[디버깅] 메모리 누수와 점검하는 방법 소개

[디버깅] 메모리 …

소개 Unmanaged Code인
C++은 메모리 할당과 해제…

luckygg.tistory.com

성능 프로파일러(Performance Profiler)

Visual Studio의 성능 프로파일러에 대해 알고 계신가요? 혹은 디버그 중에 진단 도구를 사용해본 경험이 있으신가요? Visual Studio에는 생각보다 많은 기능들을 제공합니다. 성능

프로파일러는 유용한 디버깅 도구 중 하나입니다. 물론, 버전마다 제공되는 기능에 제약이 있습니다. 버전 별 자세한 차이점은 아래 링크를 참고하세요.

https://visualstudio.microsoft.com/ko/vs/compare/

Visual Studio …

Visual Studio 2019 제품을
비교하여 가장 적합한 도…

visualstudio.microsoft.com

지원되는 기능	Visual Studio 커뮤니티 무료 다운로드	Visual Studio Professional 구입	Visual Studio Enterprise 구입
⊕ 지원 사용 시나리오	●●●○	●●●●	●●●●
개발 플랫폼 지원 [2]	●●●●	●●●●	●●●●
⊕ 통합 개발 환경	●●●○	●●●○	●●●●
⊖ 고급 디버깅 및 진단	●●○○	●●○○	●●●●
IntelliTrace			●
코드 맵 디버거 통합			●
.NET 메모리 덤프 분석기			●
코드 메트릭	●	●	●
그래픽 디버깅	●	●	●
정적 코드 분석	●	●	●
성능 및 진단 허브	●	●	●[4]
스냅숏 디버거			●
시간 이동 디버깅(미리 보기)			●
⊕ 테스트 도구	●○○○	●○○○	●●●●
⊕ 크로스 플랫폼 개발	●●○○	●●○○	●●●●
⊕ 협업 도구 및 기능	●●●●	●●●●	●●●●

Visual Studio 버전 별 차이

자, 그래서 성능 프로파일러가 무엇인지 보겠습니다. 성능 프로파일러는 메모리 및 CPU, GPU 사용량을 점검하고, 병목 현상 및 메모리 누수 등의 문제를 찾는데 유용한 기능입니다. 프로그램 개발 시 메모리 누수 문제가 제일 심각한데요. 성능 프로파일러를 이용한 메모리 누수 확인 기능은 정말 유용합니다. 성능 프로파일러에 대한 더 자세한 내용은 아래 링크를 참고하세요.

https://docs.microsoft.com/ko-kr/visualstudio/profiling/?view=vs-2019

Visual Studio…

프로파일링 도구를 사용하여
성능을 측정하는 데 Visual…

docs.microsoft.com

성능 프로파일러를 이용한 메모리 사용량 점검 방법

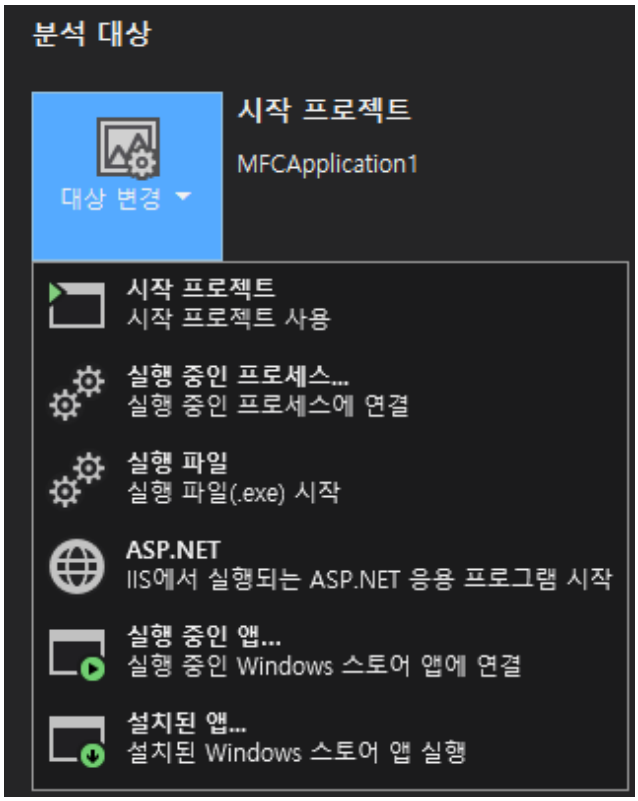
MFC Dialog 프로젝트 기반으로, 성능 프로파일러를 이용하여 메모리 사용량을 점검해 보겠습니다. 성능 프로파일러 메뉴는 Visual Studio 상단의 디버그 메뉴에 있습니다. 단축키는 `Alt + F2` 입니다.

성능 프로파일러 메뉴

성능 프로파일러를 클릭하면, 아래와 같이 분석 대상과 사용 가능한 도구가 보입니다.

성능 프로파일러 옵션

분석 대상의 변경 메뉴를 클릭하면 다음과 같이 여러 옵션이 표시됩니다. 현재 프로젝트의 메모리 사용량을 점검하려면 기본 값인 '시작 프로젝트'를 사용하면 됩니다. 그 외의 옵션들은 아래 설명과 동일하여 따로 설명이 필요하진 않겠네요.



분석 대상 옵션

이번 예제 코드에서는 Alloc 버튼을 클릭하면, 다음과 같이 메모리를 할당합니다.

```
void CMFCApplication1Dlg::OnBnClickedButton1()
{
m_pBuffer = new byte[100000];
}
```

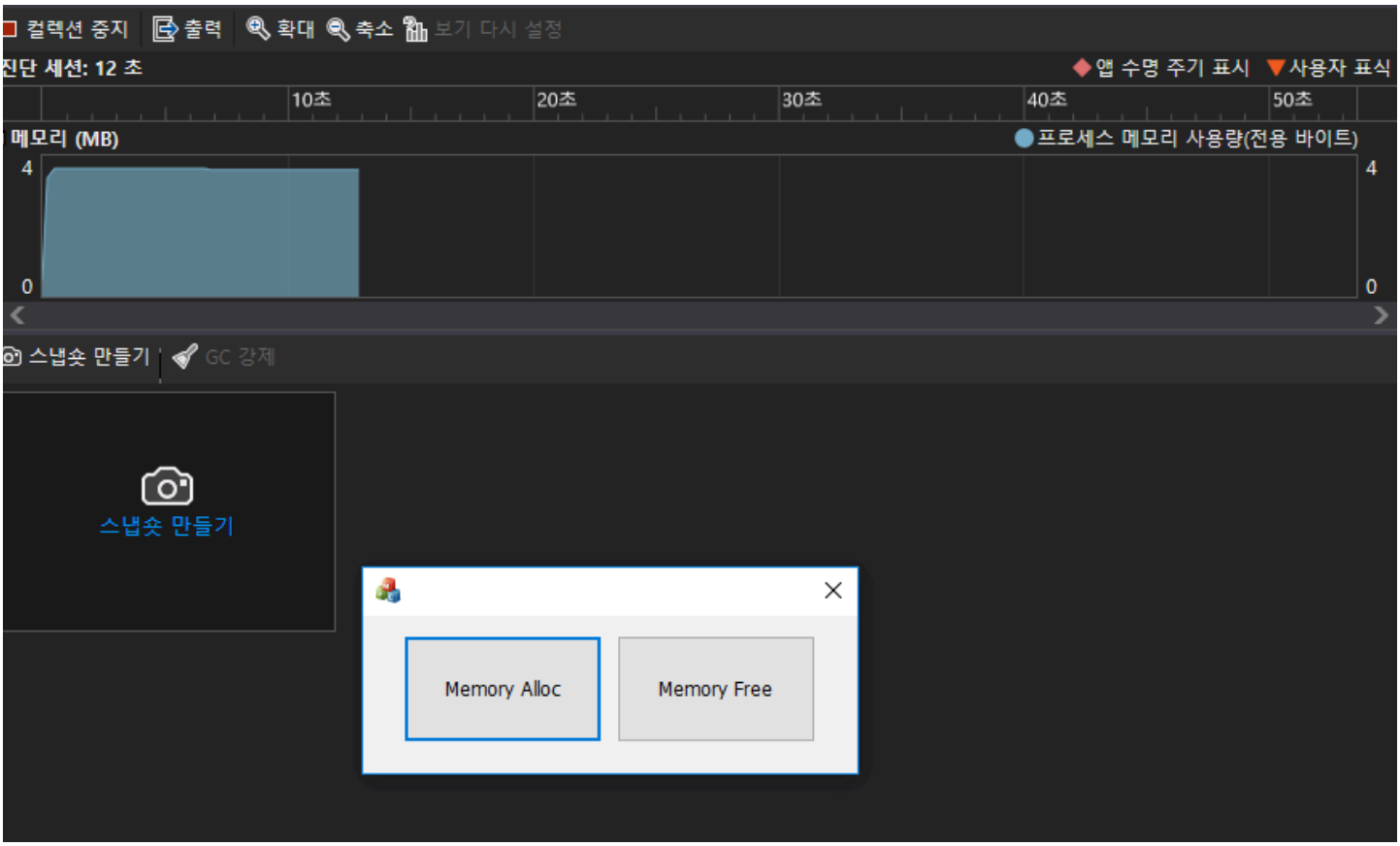
그리고 Free 버튼을 클릭하면, 다음과 같이 메모리를 해제합니다.

```
void CMFCApplication1Dlg::OnBnClickedButton2()
{
delete[]m_pBuffer;
m_pBuffer = nullptr;
}
```

성능 프로파일러 옵션에서 '사용 가능한 도구'에서 '메모리 사용량'을 체크하고 '시작' 버튼을 클릭하여 메모리 분석을 시작해 보겠습니다.

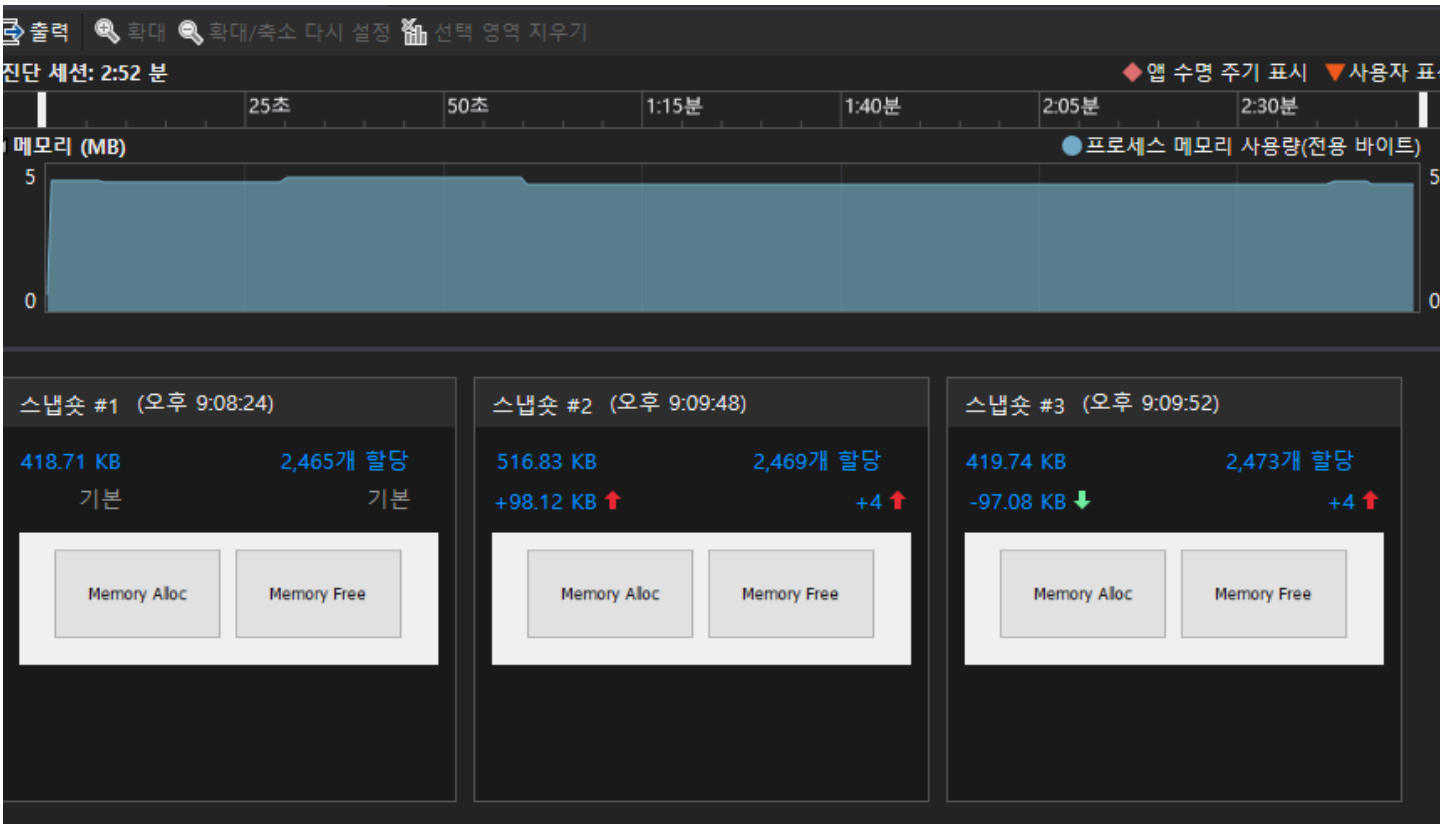
메모리 사용량 분석 시작

시작을 누르면 아래와 같은 화면이 보여집니다. 시간에 따른 메모리 사용량 그래프가 보이고, 아래에는 스냅숏 만들기라는 버튼이 보입니다.



메모리 사용량 분석 중인 화면

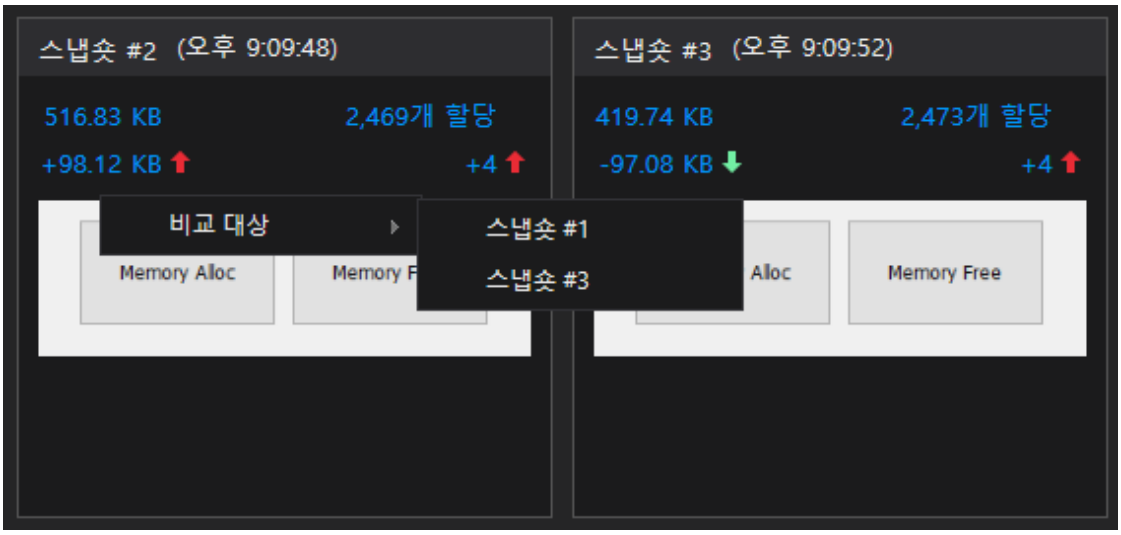
'스냅숏 만들기'버튼을 클릭하면, 현재 메모리 사용량 정보를 저장합니다. 본문에서는 메모리 할당 전, 할당 후, 해제 후의 스냅숏 정보를 저장하도록 하겠습니다.



메모리 사용량 분석 결과 화면

메모리 할당 전에는 418.71KB이고, 할당 후에는 98.12KB가 증가한 516.83KB로 표시됩니다. 그리고 해제 후에는 97.08KB가 감소한 419.74KB가 됩니다. 메모리 할당 전과 해제 후의 사용량이 다릅니다. 마치 메모리가 모두 해제가 안된 듯한 모습인데요. 이는 본문 하단에서 다시 설명하겠습니다.

스냅숏 간의 좀 더 자세한 정보를 취득하기 위해서는, 특정 스냅숏에서 우측 마우스를 클릭하여 다른 스냅숏과 비교할 수 있습니다.



스냅숏 비교 메뉴

스냅숏 #1과 #2의 비교 결과를 보겠습니다. 예시 프로젝트의 이름이 MFCApplication1이므로, 해당 이름과 관련된 식별자를 찾아보았습니다. 찾은 결과 `CMFCApplication1Dlg::OnBnClickedButton1`에서 `100,036Byte`가 증가한 모습이 보입니다.

이터브 힙

보기 설정 필터가 적용되었습니다(내 코드만). 힙 콘텐츠 보기 호출 스택 집계 기준: 호출자

별자	개수	크기(바이트)	개수 차이	크기 차이(바이트)	모듈
	2,469	529,230	+4	+100,476	
[외부 코드]	2,469	529,230	+4	+100,476	[외부 코드]
_wWinMainCRTStartup	1,790	351,002	+4	+100,476	MFCApplication1.exe
WinbaseIsolationAwarePrivatetRgzlnPgpgk	1,790	351,002	+4	+100,476	MFCApplication1.exe
WinbaseIsolationAwarePrivatetRgzlnPgpgk	1,790	351,002	+4	+100,476	MFCApplication1.exe
__scrt_wide_environment_policy::initialize_environment	1,719	339,356	+4	+100,476	MFCApplication1.exe
wWinMain	1,719	339,356	+4	+100,476	MFCApplication1.exe
[외부 코드]	1,719	339,356	+4	+100,476	[외부 코드]
CMFCApplication1App::InitInstance	1,716	339,060	+4	+100,476	MFCApplication1.exe
[외부 코드]	1,716	339,060	+4	+100,476	[외부 코드]
CMFCApplication1Dlg::OnBnClickedButton1	1	100,036	+1	+100,036	MFCApplication1.exe
[외부 코드]	1	100,036	+1	+100,036	[외부 코드]
[win32kfull.sys의 알 수 없는 프레임]	480	99,622	+3	+440	win32kfull.sys
[자동]	1,231	138,562	0	0	
CMFCApplication1Dlg::OnInitDialog	4	840	0	0	MFCApplication1.exe
[자동]	3	296	0	0	
[외부 코드]	71	11,646	0	0	[외부 코드]
[자동]	623	162,028	0	0	
[win32kfull.sys의 알 수 없는 프레임]	42	13,612	0	0	win32kfull.sys
[win32kbase.sys의 알 수 없는 프레임]	14	2,588	0	0	win32kbase.sys

할당

식별자	주소	개체 형식	크기(바이트)	모듈
CMFCApplication1Dlg::OnBnClickedButton1	0xc5ff80		100,036	MFCApplication1.exe

스냅숏 비교 결과

우측 상단의 '힙 콘텐츠 보기'를 클릭하면 아래와 같은 화면이 표시됩니다. `unsigned char[]`로 `100,000Byte`의 크기 차이가 있다고 보입니다.

본 메모리(MFCApplication1.exe) 비교 대상: 스냅숏 #1

개체 형식	개수 차이	크기 차이(바이트)	새 할당 개수	개수	크기(바이트)
unsigned char[]	+1	+100,000	+1	1	100,000
MFCApplication1.exe!CShellManager	0	0	0	1	12

힙 표시 화면

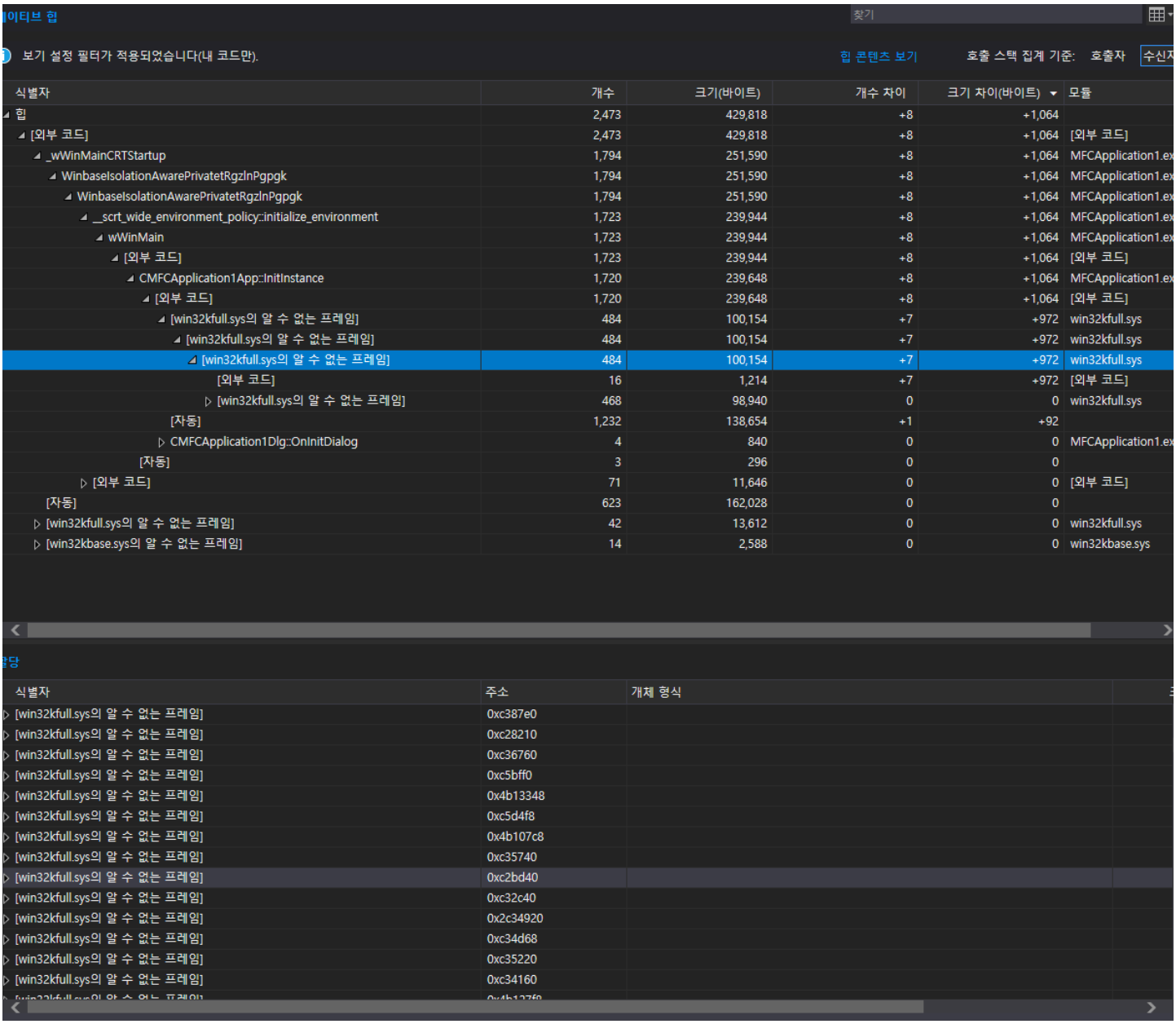
같은 방법으로, 스냅숏 #2와 #3의 비교를 설정한 후 메모리 감소량을 분석해보면 아래의 화면이 표시됩니다.

본 메모리(MFCApplication1.exe) 비교 대상: 스냅숏 #2

개체 형식	개수 차이	크기 차이(바이트)	새 할당 개수	개수	크기(바이트)
MFCApplication1.exe!CShellManager	0	0	0	1	12
unsigned char[]	-1	-100,000	0	0	0

힙 표시 화면

자, 그럼 사용자가 생성하고 해제한 메모리는 말끔히 지워진 것을 확인했습니다. 그런데 왜 전체 프로그램의 메모리 사용량에는 차이가 있을까요? 위에서 소개한 같은 방법으로 추적해 보았더니 `win32kfull.sys`라는 곳에서 메모리 사용이 증가한 것을 확인했습니다. 아쉽지만, 더 이상 의미 있는 추적은 불가능했습니다. 알 수 없는 프레임이라는 정보만 표시되고, 힙 콘텐츠에도 정보가 표시되지 않습니다.



win32kfull.sys에서 메모리 사용이 증가한 모습

결론

MFC Dialog 프로젝트 기준으로, `win32kfull.sys`에서의 메모리 사용량 증가에 대해서는 명확히 알 수 없었지만, 적어도 사용자가 생성하고 해제한 메모리의 점검은 가능했습니다. 이전 포스팅에서 설명한 CRT 라이브러리보다 사용하기도 편리하고, 문제점을 찾는 방법도 더욱 쉽다는 것도 알 수 있었습니다. 본문에서는 메모리 사용량 점검만 했지만, CPU 및 GPU 점검도 가능하고, 프로젝트뿐만 아니라 실행 파일 또는 프로세스도 분석도 가능하다는 점 알고 계시기 바랍니다.